

Corporate Risk Intelligence Platform

Product & Technical Documentation

Version: 1.0 | **Status:** MVP Design Phase | **Initial Company:** Tata Motors

1. What This Product Does

The Corporate Risk Intelligence Platform reads a company's annual reports across multiple years and tells you **what actually changed in its risk profile — and why it matters** — instead of just flagging that some text changed.

Core question it answers:

"What changed in this company's risk profile compared with previous years, and why does it matter?"

Why a text-diff isn't enough

FY24 disclosure	FY25 disclosure	Plain text-diff sees	This system sees
"Cybersecurity incidents <i>may</i> adversely affect our operations."	"We <i>experienced</i> cybersecurity incidents that resulted in operational disruptions."	Some words changed	Risk status moved from potential exposure → active/realized risk

A basic diff tool can't reason about business meaning. An LLM-assisted semantic pipeline can.

2. Business Problem

Public companies publish long annual reports with risk disclosures. Comparing several years of these by hand is slow and error-prone:



Who needs this

Financial advisory — due diligence, valuation, transaction advisory, risk advisory

Investment — equity research, investment analysts, PE/VC, credit analysts

Audit — audit teams, forensic accounting

Corporate — enterprise risk teams, strategy, investor relations

3. Scope

Geography (initial): India

Document type: Annual reports of listed Indian companies

Development company: Tata Motors

Period: 4 years (FY22-23 → FY25-26)

Architecture goal: designed from day one to generalize to multiple companies, years, and industries — even though MVP is single-company

4. End-to-End Workflow



Select Company → Select Reporting Years → Retrieve Annual Reports
→ Document Processing → Identify Risk Disclosures → Extract Individual Risks
→ Normalize Risks → Generate Embeddings → Semantic Matching
→ Compare Risk Pairs → LLM Substantive Analysis
→ Risk Change Classification → Risk Impact Scoring → Analyst Report

5. Risk Change Taxonomy

Every risk comparison between two years falls into exactly one bucket:

Category	Meaning
New	Appears in the latest report; no historical match
Dropped	Existed before; no corresponding disclosure now
Stable	Underlying risk substantially unchanged
Minor change	Wording changed, risk itself is the same
Materially changed	Nature, severity, scope, likelihood, or status meaningfully changed

What "materially changed" is evaluated across

Risk status progression: Hypothetical → Emerging → Active → Realized

Severity: Low → Medium → High → Critical

Scope: Narrower / Same / Broader

Business impact areas: revenue, margins, cash flow, operations, compliance, reputation, capital requirements, supply chain, valuation

Management response: no change / new mitigation / expanded mitigation / reduced mitigation

Careful language matters

A newly-appearing risk is called a **"newly disclosed risk,"** not a **"new risk to the company"** — the company may have faced it before without disclosing it.

A risk with no current-year match is a **"potentially dropped risk"** flagged for analyst review — the system never asserts "the risk no longer exists."

6. Standardized Risk Categories

The system maps each raw disclosure to a standard category so risks are comparable across companies:

Financial · Operational · Regulatory · Legal/Litigation · Cybersecurity · Technology · Supply Chain · Market · Competition · Geopolitical · Environmental · Climate · Commodity · Foreign Exchange · Interest Rate · Liquidity · Reputation · Human Capital

7. Technical Architecture

7.1 Data sources

Company Investor Relations websites, NSE/BSE filings, SEBI disclosures

Explicitly avoided: Kaggle datasets, fabricated data, financial blogs, scraped summaries — primary-source documents only

7.2 Document ingestion


PDF Annual Report → Document parser (Docling, PyMuPDF, OCR fallback) → Page structure → Sections → Risk-related sections → Risk units

Risk sections aren't assumed to share a heading — the system looks across "Risk Management," "Enterprise Risk Management," "Principal Risks," "MD&A," and other variants.

7.3 Extracted risk record (example)


json

```
{
  "company": "Tata Motors",
  "fiscal_year": "2025",
  "risk_id": "risk_023",
  "category": "Supply Chain",
  "title": "...",
  "text": "...",
  "page": 142,
  "source_document": "FY2025 Annual Report"
}
```

7.4 Normalization

Differently-worded disclosures describing the same underlying risk (e.g. "dependence on semiconductor suppliers," "availability of critical electronic components," "global semiconductor shortages") are reconciled using: risk title, category, entities, events, geography, business unit, severity indicators, temporal indicators.

7.5 Embeddings + vector search

Each risk factor → embedding vector (candidate models: **BGE**, **Nomic**, or other strong open-source embedding models)

Qdrant as the vector store — chosen for being open source, having good Python support, metadata filtering, semantic search, and easy local deployment

Qdrant is a search index, **not** the system of record

7.6 Two-stage semantic matching


Stage 1 — Vector retrieval: 2025 Risk → Qdrant → Top 5 candidates from 2024
Stage 2 — LLM validation: same underlying risk? yes/no + confidence (0-1) + reason

This avoids relying on cosine similarity alone. Match thresholds are set through evaluation, not hardcoded guesses.

7.7 System of record

PostgreSQL stores: companies, filings, reporting periods, documents, risk factors, risk categories, risk matches, change classifications, analysis results, users, reports.


Company → Filings → Risks → Comparisons

7.8 LLM reasoning layer

Input to the LLM: historical risk + latest risk + metadata + relevant context.

Example structured output:

<pre> json { "classification": "material_change", "risk_status_change": "potential_to_active", "severity_change": "increased", "scope_change": "expanded", "explanation": "...", "business_implication": "...", "confidence": 0.91 }</pre>

Provider abstraction: the app doesn't depend directly on any one LLM provider —

Ollama (local, open-source models) for development

Groq (hosted open-weight models) for fast cloud inference / deployment

Guardrails: the model must only use supplied evidence, avoid inventing financial facts, distinguish inference from disclosed fact, always return structured JSON with a confidence score and source references, and explicitly say "insufficient evidence" when appropriate.

7.9 Evidence & citation system

Every conclusion is traceable:

AI conclusion → Comparison → Source risk → Original annual report → Page number

This traceability is treated as non-negotiable for a financial application.

7.10 Risk Change Score (future)

A single 0–100 score (Low → Medium → High) combining semantic change, severity change, status change, scope change, event evidence, and management response — formula to be validated against historical examples, not invented upfront.

7.11 Graph layer (future — Neo4j, not MVP)

Company —operates in→ Industry Company —exposed to→ Risk Company —affected by→ Regulation Company —experienced→ Event Company —operates in→ Geography

Enables future cross-company questions like "which risks are increasing across automotive companies?"

7.12 Async processing

User → FastAPI → Create analysis job → Redis → Celery worker
→ Document processing → Embedding → LLM analysis → Database

UI shows a "Processing... 35%" style progress state.

7.13 Object storage

Raw and processed documents are kept separately for reproducibility:

/raw/tata_motors/{FY2023,FY2024,FY2025}.pdf
/processed/tata_motors/{FY2023,FY2024,FY2025}.json

7.14 High-level architecture diagram

USER → Gradio → FastAPI → [Redis + PostgreSQL]

↓

Celery → Ingestion Service

↓

S3/MinIO + Docling/PyMuPDF

↓

Risk Extraction

↓

Qdrant + PostgreSQL

↓

Matching Engine

↓

LLM Service (Ollama / Groq)

↓

Change Analysis

↓

Risk Intelligence → Report/UI

7.15 Technology stack summary

Layer	Technology
Frontend	Gradio (React planned later)
Backend	FastAPI (Python)
LLM — local	Ollama

Layer	Technology
LLM — cloud	Groq
Embeddings	BGE / Nomic
Vector DB	Qdrant
Database	PostgreSQL
Cache	Redis
Background jobs	Celery
Document parsing	Docling / PyMuPDF
Object storage	S3 / MinIO
Graph DB	Neo4j <i>(future)</i>
Containerization	Docker
CI/CD	GitHub Actions
Monitoring	Prometheus/Grafana <i>(future)</i>

8. API Design (Initial Set)

Method	Endpoint	Purpose
POST	<code>/companies</code>	Add a company
GET	<code>/companies</code>	List companies
POST	<code>/filings/upload</code>	Upload an annual report
GET	<code>/filings/{company}</code>	List filings for a company
POST	<code>/analysis/run</code>	Kick off a comparison run
GET	<code>/analysis/{analysis_id}</code>	Get analysis status/results
GET	<code>/risks/{company}</code>	List extracted risks
GET	<code>/reports/{company}</code>	Get the analyst report

Example request:

9. Product Output — Example

Executive summary

Total risks analyzed: 31
New risks: 4
Dropped risks: 2
Stable risks: 18
Materially changed: 7
High-priority changes: 3

Individual risk analysis (example)

Risk Category: Cybersecurity
FY24: "..."
FY25: "..."
Classification: Materially Changed
Change: Potential → Active
Severity: Increased
Why it matters: ...
Evidence: Annual Report FY24 (page XX), Annual Report FY25 (page XX)

Gradio UI sketch

Corporate Risk Intelligence
Company: [Tata Motors ▼] Compare: [FY2024] → [FY2025] [Run Analysis]
Risk Summary — New 4 Dropped 2 Stable 18 Material 7
High Priority Changes
● Cybersecurity ● Supply Chain ● Regulatory

Clicking a risk shows: FY24 text → FY25 text → AI explanation → business implication → evidence.

10. Evaluation Framework

Build a human-labeled benchmark of risk pairs, then measure:

Component	Metrics
Matching (same risk Y/N)	Precision, Recall, F1
Classification (new/dropped/stable/minor/material)	Accuracy, Precision, Recall, F1
LLM explanations	Human-rated: correct? grounded? useful? hallucination?

11. Production Quality Requirements

Reliability: retry failed jobs; handle malformed PDFs, missing sections, LLM failures

Observability: log document_id, company, filing_year, processing_time, model, tokens, errors, confidence

Reproducibility: same document + same configuration → traceable, repeatable results

Versioning: track embedding_model, LLM_model, prompt_version, pipeline_version

12. Security

MVP: public documents only → low sensitivity

Future enterprise deployment: authentication, RBAC, tenant isolation, encryption, audit logging, private/VPC/on-prem deployment options, with per-client data isolation (Client A/B/C each scoped to their own company data)

13. MVP vs. Production Target

MVP

☐ Tata Motors → 4 annual reports → risk extraction → embeddings
→ Qdrant → matching → Ollama → Gradio

Production target

☐

14. Development Roadmap

Phase	Focus	Key deliverable
1 — Finance + Data	Understand & extract real risk disclosures	Structured risks for Tata Motors, 4 reports
2 — Semantic Retrieval	Embeddings + Qdrant	Candidate matching across years
3 — AI Reasoning	Ollama-driven analysis	Material-change classification
4 — Product	FastAPI + Gradio + PostgreSQL	Working end-to-end app
5 — Production Infra	Redis + Celery + S3 + Docker + logging	Scalable, observable pipeline
6 — Advanced Intelligence	Neo4j, cross-company analysis, trend detection	Industry-level risk benchmarking

Explicit build discipline: don't move to embeddings/Qdrant until Phase 1 (extraction & structuring) output is validated on real Tata Motors data.

15. Long-Term Vision

The product is designed to grow from:

"Compare two annual reports."

toward:

"Understand how corporate risk is evolving" — company risk trends, industry/peer comparison, and multi-year trend analysis, ultimately answering questions like *"Which risks are becoming more important across the automotive industry, which companies are most exposed, and what evidence supports that conclusion?"*

But the near-term target stays deliberately narrow: **Tata Motors → 4 real annual reports → correctly identified risk disclosures → clean structured data.**